Generating evenly distributed points on spheres

authors

April 2022

Given a $d-1$-sphere $\mathcal{S}^{d-1} = \{\boldsymbol{x} \in \mathbb{R}^d : \|\boldsymbol{x}\| = 1\}$, we want to generate a number of points n, $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n$ where each $\boldsymbol{x}_i \in \mathbb{R}^d$ on this sphere $\mathcal{S}^{d-1}$. These points $\boldsymbol{x}_i$ should have the *equidistant property*: The distance between points and their neighbors are as even as possible; and *maximized minimal distance*: the minimum distance between any two points to be as large as possible. Let $\mathcal{D}_n = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n\}$. We have the following optimization problem

$$\max_{\mathcal{D}_n} \sum_{\boldsymbol{x}_i, \boldsymbol{x}_j \in \mathcal{D}_n, i \neq j} \|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2 \tag{1}$$

1 Generalized Fibonacci sphere

Definition 1 (Fibonacci lattice)**.** *n evenly distributes points inside a unit square $[0, 1)^2$ are*

$$\boldsymbol{x}_i = (x_1, x_2) = \left(\frac{i}{\phi}\%1, \frac{i}{n}\right) \quad \textit{for } 0 \leq i < n \tag{2}$$

where $\phi = \frac{1+\sqrt{5}}{2} = \lim_{n\to\infty}\left(\frac{F_{n+1}}{F_n}\right)$. the $x\%1$ operator denotes the fractional part of the number x.

Example of $n = 50, 100, 200$ points of Fibonacci points are illustrated in Figure 1.

To get Fibonacci spiral, we simply map this point distribution to a unit disk, via the equal-area transformation. This transformation can be seen in Figure 2.

$$\boldsymbol{x}_i = (x_i(1), x_i(2)) \to (\theta, r) : (2\pi x_i(1), \sqrt{x_i(2)}) \tag{3}$$

Generalization. Fibonacci lattice is based on the fact that when distributed uniformly, both the angle and latitude sine are uniform. Let's use the generalized polar coordinate system.

$$(r, \theta_1, \theta_2, \ldots, \theta_{d-1}) \tag{4}$$

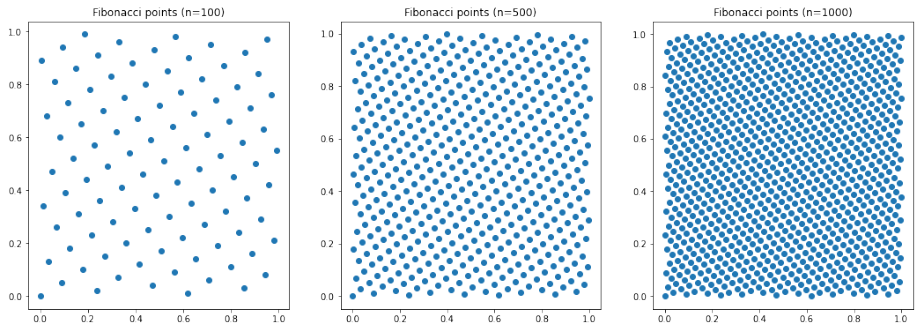

Figure 1: Fibonacci points

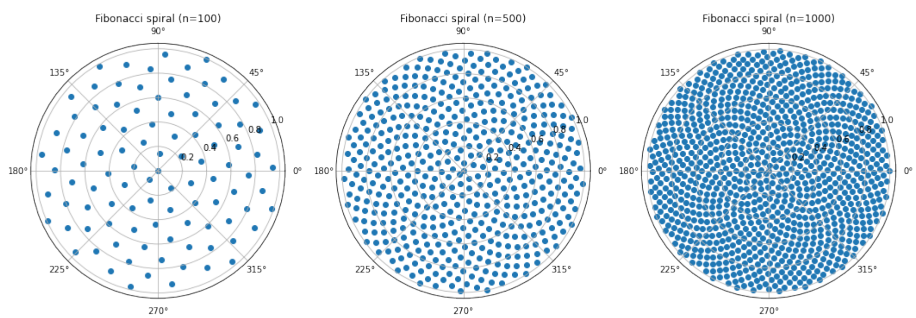

Figure 2: Fibonacci spiral

For $i \in [n]$, generate the points $\boldsymbol{x}_i$ with

$$\begin{aligned} x_i(1) &= n \cdot a_{d-1} \\ x_i(2) &= n \cdot a_{d-2} \\ &\cdots \\ x_i(d-1) &= n \cdot a_1 \\ x_i(d) &= \frac{i}{n+1}, \end{aligned}$$

where $a_i, a_j \in \mathbb{Q} \forall i \neq j$. Example of a_i could be $\sqrt{2}, \sqrt{3}, \sqrt{7}, \ldots$.

Algorithm 1 GenFibonacci(n, d)

Input: n, d
for $i = 1, 2, \ldots, n$ **do**
    $\boldsymbol{x}_i = \mathbf{1}$
for $i = 1, 2, \ldots, n$ **do**
    $t = \frac{2\pi i}{n}$
    $\boldsymbol{x}_i[0] = x_i[0] \times \sin(t)$
    $\boldsymbol{x}_i[1] = x_i[1] \times \cos(t)$
Let $\boldsymbol{p} = [2, 3, 5, 7, 11, 13, \ldots]$ be list of prime numbers
for $d_i = 2, 3, \ldots, d-1$ **do**
    Let $f(m) = \frac{\Gamma(\frac{d_i+1}{2})}{\Gamma(\frac{d_i}{2}) \cdot \sqrt{\pi}} \int \sin^m(t) dt$
    for $j = 1, 2, \ldots, n$ **do**
        $\theta_i = f^{-1}(\{j * \sqrt{p_i}\})$
        for $k = 1, 2, \ldots, d-1$ **do**
            $\boldsymbol{x}_i[k] = \boldsymbol{x}_i[k] * \sin(\theta_i)$
        $\boldsymbol{x}_i[d] = \boldsymbol{x}_i[d] * \cos(\theta_i)$
Return $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n$

2 Optimizing minimum nearest-neighbor distance

The problem can also be treated as optimizing minimum nearest-neighbor distance, which is similar to Tammes problem. In geometry, the Tammes problem is a problem in packing a given number of circles on the surface of a sphere such that the minimum distance between circles is maximized.

If, for each of the n points, we define δ_i to be distance from point i to its closest neighbor ($0 \leq i < n$). Then Tammes's problem asks us to maximize the minimum of δ_i over all n points. That is,

$$\max_{\mathcal{D}_n} \left\{ \delta_{min}(n) := \min_{0 \leq i < n} \delta_i \right\}, \text{ where } \delta_i = \min_{j \neq i} \|\boldsymbol{x}_i - \boldsymbol{x}_j\| \tag{5}$$

The above formulation is extremely fundamental and this problem is unsolved for large number of points. — (ref. *Alexandre Eremenko*).

3 Remove points to maximize shortest nearest neighbor distance

Suppose we generate m points and we want to remove $m - n$ points via the following heuristic ways:

- Find the closest two points, remove the point that has the smallest next nearest neighbor distance, repeat until we have n points left.
- Calculate all the pairwise distances once, sort all pairs by distance, and remove points from shortest pairs that are in fewest number of pairs

There are many equally valid criteria that are used to describe how evenly distributed the points on the surface of a sphere are. These include:

- Packing and covering
- Convex hulls, Voronoi cells and Delaunay triangles

It is the fact that an optimal solution based on one criteria is often not an optimal point distribution for another.

4 Ideas of moving points

See [1]. Link: https://www.osti.gov/servlets/purl/1571374

[1] Yao, J. An Equal-distance Smoother for a Point Set Distributed in an Arbitrary Region. No. LLNL-CONF-779421. Lawrence Livermore National Lab.(LLNL), Livermore, CA (United States), 2019.

5 Tasks

We want to see how the cone variance analysis works on real data sets. For example, we can generate 100/1000 data points in dimension 100 as centers. For each four word embedding sets, counts how many points are in each cone. Plot the frequency distribution. Report the variance of data points at each center.

We cam repeat the above procedure X times with different but equivilent cone sets (perhaps by rotation) and see how consistent the results are.